SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Course Code:	**CSA12**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO5**	**Assessment:**	**ASSESSMENT TOOL2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)**
Weightage:	**30%**	**Total Marks:**	**25**
CO5 Statement:	*Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.*		
Student Name:	P. Ahlen Abishek	**Reg.No.:**	192311034
Department:	CSE- dep	**Date:**	26 – 08 – 26

80.1/8

ANNEXURE A
Pseudocode Writing Task (Algorithm Design)

1. Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.
2. Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.
3. Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.
4. Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.
5. Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Code of Conduct:

I P. Ahlen Abishek *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (20%)	Algorithm Design (30%)	Use of Control Structures (20%)	Pseudocode Structure (10%)	Completeness (20%)
Q1	2	2	2	1	2
Q2	2	2	1.5	1	2
Q3	1.5	1.5	1.5	0.5	1.5
Q4	1.5	2	1.5	0.5	1.5
Q5	1	3	2	1	2

AO

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20% 2	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30% 3	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20% 2	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10% 1	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20% 2	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

SIMATS ENGINEERING

Department of Computer Science & Engineering

Course Code: CSA12 | Course Title: Computer Architecture | CO Assessed: CO5

ASSESSMENT TOOL 2 – PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Student Name: Arena Abishek

Reg. No.: 192311034

Department: Computer Science & Engineering

Date: _____

Q1. DMA-Based Data Transfer Algorithm (with interrupt handling, minimal CPU involvement)

Problem Understanding:

Input: I/O device, memory base address, transfer length. Output: data moved from device to memory with the CPU freed during the bulk transfer and notified only on completion via interrupt.

```
ALGORITHM DMA_Transfer(device, memory_base_addr, transfer_length)
BEGIN
    // Step 1: CPU initializes DMA controller registers, then steps away
    DMA_CONTROLLER.set_source(device.data_register)
    DMA_CONTROLLER.set_destination(memory_base_addr)
    DMA_CONTROLLER.set_length(transfer_length)
    DMA_CONTROLLER.set_mode(BURST)

    DMA_CONTROLLER.START()
    CPU.release_bus()
    CPU.continue_other_tasks()          // CPU is free during transfer

    // Step 2: DMA controller performs the transfer autonomously
    WHILE DMA_CONTROLLER.bytes_remaining > 0 DO
        REQUEST_BUS()
        IF BUS_GRANTED THEN
            TRANSFER_WORD(device.data_register, memory_base_addr)
            memory_base_addr = memory_base_addr + WORD_SIZE
            DMA_CONTROLLER.bytes_remaining -= WORD_SIZE
            RELEASE_BUS()
        END IF
    END WHILE

    // Step 3: Notify CPU only when done
    RAISE_INTERRUPT(DMA_COMPLETE_IRQ)
END

INTERRUPT_HANDLER DMA_Complete_ISR()
BEGIN
    SAVE_CPU_CONTEXT()
```

```
    IF DMA_CONTROLLER.status == SUCCESS THEN
        NOTIFY_OS("Transfer complete")
        MARK_BUFFER_READY()
    ELSE
        LOG_ERROR("DMA transfer failed")
        RETRY_OR_ABORT()
    END IF
    RESTORE_CPU_CONTEXT()
    RETURN_FROM_INTERRUPT()
END
```

Q2. Interrupt Handling System — Vectored Priority Dispatch

Problem Understanding:

Multiple devices raise interrupts concurrently; each must be identified via its vector, ranked by priority, and dispatched to the correct service routine without losing lower-priority requests.

```
GLOBAL interrupt_pending_queue[]        // priority queue
GLOBAL VECTOR_TABLE[MAX_DEVICES]        // vector -> ISR address

ALGORITHM Vectored_Interrupt_Handler()
BEGIN
    WHILE SYSTEM_RUNNING DO
        WAIT_FOR_INTERRUPT_SIGNAL()
        IF INTERRUPT_LINE_ACTIVE THEN
            device = IDENTIFY_INTERRUPTING_DEVICE()   // via interrupt controller
            interrupt_pending_queue.INSERT(device, device.priority)
        END IF

        WHILE interrupt_pending_queue.NOT_EMPTY() DO
            current = interrupt_pending_queue.EXTRACT_MAX_PRIORITY()
            IF current.priority > CURRENTLY_EXECUTING_PRIORITY THEN
                SAVE_CPU_CONTEXT()
                DISABLE_LOWER_PRIORITY_INTERRUPTS(current.priority)
                isr_address = VECTOR_TABLE[current.vector]
                JUMP_TO(isr_address)
                EXECUTE_ISR(current)
                ENABLE_LOWER_PRIORITY_INTERRUPTS()
                RESTORE_CPU_CONTEXT()
            ELSE
                interrupt_pending_queue.INSERT(current, current.priority)
                BREAK
            END IF
        END WHILE
    END WHILE
END

FUNCTION EXECUTE_ISR(device)
BEGIN
    SWITCH device.type:
        CASE KEYBOARD: CALL Keyboard_ISR()
        CASE DISK:     CALL Disk_ISR()
        CASE NETWORK:  CALL Network_ISR()
        DEFAULT:       CALL Generic_ISR(device)
    END SWITCH
    ACKNOWLEDGE_INTERRUPT(device)
```

```
    RETURN_FROM_INTERRUPT()
END
```

Q3. Dynamic Selection: Memory-Mapped I/O vs I/O-Mapped I/O

Problem Understanding:

Decide, per device request, which addressing scheme minimises latency while meeting bandwidth needs, using measured performance rather than a fixed choice.

```
ALGORITHM Select_IO_Mechanism(device, required_latency, required_bandwidth)
BEGIN
    lat_MMIO = BENCHMARK_LATENCY(MEMORY_MAPPED)
    lat_IOM  = BENCHMARK_LATENCY(IO_MAPPED)
    bw_MMIO  = BENCHMARK_BANDWIDTH(MEMORY_MAPPED)
    bw_IOM   = BENCHMARK_BANDWIDTH(IO_MAPPED)

    IF required_bandwidth > HIGH_BANDWIDTH_THRESHOLD THEN
        chosen = MEMORY_MAPPED        // full instruction set + caching + DMA-friendly
    ELSE IF required_latency < LOW_LATENCY_THRESHOLD THEN
        IF lat_IOM < lat_MMIO THEN
            chosen = IO_MAPPED         // dedicated bus lines, less contention
        ELSE
            chosen = MEMORY_MAPPED
        END IF
    ELSE
        IF bw_MMIO >= required_bandwidth AND lat_MMIO <= required_latency THEN
            chosen = MEMORY_MAPPED
        ELSE IF bw_IOM >= required_bandwidth AND lat_IOM <= required_latency THEN
            chosen = IO_MAPPED
        ELSE
            chosen = MEMORY_MAPPED     // safe default
        END IF
    END IF

    CONFIGURE_DEVICE_ACCESS(device, chosen)
    RETURN chosen
END
```

Q4. Bus Arbitration Across PCIe, USB and Thunderbolt

Problem Understanding:

Several devices on different bus standards compete for a shared interconnect; arbitration must grant access fairly, respect each bus's throughput class, and avoid starving lower-priority devices.

```
GLOBAL bus_owner = NULL
GLOBAL request_queue[]

ALGORITHM Bus_Arbitration()
BEGIN
    WHILE SYSTEM_RUNNING DO
        FOR EACH device IN CONNECTED_DEVICES DO
            IF device.request_pending THEN
                request_queue.INSERT(device, COMPUTE_PRIORITY(device))
            END IF
        END FOR
```

```
        IF bus_owner == NULL AND request_queue.NOT_EMPTY() THEN
            bus_owner = request_queue.EXTRACT_MAX_PRIORITY()
            GRANT_BUS(bus_owner)
            START_TIMER(bus_owner, MAX_HOLD_TIME(bus_owner.bus_type))
        END IF

        IF bus_owner != NULL AND (bus_owner.transfer_complete OR TIMER_EXPIRED()) THEN
            RELEASE_BUS(bus_owner)
            bus_owner = NULL
        END IF
    END WHILE
END

FUNCTION COMPUTE_PRIORITY(device)
BEGIN
    SWITCH device.bus_type:
        CASE Thunderbolt: base = HIGH
        CASE PCIe:        base = MEDIUM_HIGH
        CASE USB:         base = MEDIUM
    END SWITCH
    waiting_time = CURRENT_TIME() - device.request_timestamp
    RETURN base + (AGING_FACTOR * waiting_time)   // aging prevents starvation
END

FUNCTION MAX_HOLD_TIME(bus_type)
BEGIN
    SWITCH bus_type:
        CASE Thunderbolt: RETURN T_HIGH
        CASE PCIe:        RETURN T_MED
        CASE USB:         RETURN T_LOW
    END SWITCH
END
```

Q5. Hybrid I/O: Polling (Low-Speed) + DMA/Interrupts (High-Speed)

Problem Understanding:

Low-speed devices are cheaply serviced by polling; high-speed devices need DMA with interrupt-on-completion to avoid CPU overload. Both must run concurrently.

```
ALGORITHM Hybrid_IO_Manager(device_list)
BEGIN
    FOR EACH device IN device_list DO
        device.speed_class = (device.data_rate < SPEED_THRESHOLD) ? LOW : HIGH
    END FOR
    START_THREAD(Polling_Loop, FILTER(device_list, LOW))
    START_THREAD(DMA_Interrupt_Loop, FILTER(device_list, HIGH))
END

PROCESS Polling_Loop(low_speed_devices)
BEGIN
    WHILE SYSTEM_RUNNING DO
        FOR EACH device IN low_speed_devices DO
            IF READ_STATUS_REGISTER(device) == DATA_READY THEN
                data = READ_DATA_REGISTER(device)
                PROCESS_DATA(device, data)
            END IF
        END FOR
```

```
        SLEEP(POLL_INTERVAL)
    END WHILE
END

PROCESS DMA_Interrupt_Loop(high_speed_devices)
BEGIN
    FOR EACH device IN high_speed_devices DO
        CONFIGURE_DMA_CHANNEL(device, device.buffer, device.transfer_size)
        DMA_CONTROLLER.START(device)
    END FOR

    WHILE SYSTEM_RUNNING DO
        WAIT_FOR_INTERRUPT()
        d = IDENTIFY_DEVICE()
        SAVE_CPU_CONTEXT()
        HANDLE_DMA_COMPLETE_ISR(d)
        RESTORE_CPU_CONTEXT()
        CONFIGURE_DMA_CHANNEL(d, next_buffer(d), d.transfer_size)
        DMA_CONTROLLER.START(d)          // re-arm for continuous stream
    END WHILE
END
```

Certification: I, Arena Abishek, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

Signature of the student: _____